

1c_bonus-F.md

Here's a CloudWatch Logs Insights query pack you can drop straight into the Lab 1C-Bonus-B incident runbook.

Two important notes up front:

CloudWatch Logs Insights works only on logs that are in CloudWatch Logs.

So this pack covers:

WAF logs (when you chose `waf_log_destination="cloudwatch"`)

App logs (your `/aws/ec2/<prefix>-rds-app` group)

ALB access logs are in S3, not CloudWatch Logs (unless you also ship them to CW via another pipeline).

For ALB, you'll correlate via:

CloudWatch metrics (5xx alarm + metrics)

and optionally Athena later (if you want the full CBRE-style "log lake" workflow)

Lab 1C-Bonus-F: Logs Insights Query Pack

Variables students fill in (for the runbook)

WAF log group: `aws-waf-logs-<project>-webacl01`

App log group: `/aws/ec2/<project>-rds-app`

Requirements: Set the time range to Last 15 minutes (or match incident window).

A) WAF Queries (CloudWatch Logs Insights)

A1) "What's happening right now?" (Top actions: ALLOW/BLOCK)

fields `@timestamp, action`

| stats count() as hits by action

| sort hits desc

A2) Top client IPs (who is hitting us the most?)

```
fields @timestamp, httpRequest.clientIp as clientIp
| stats count() as hits by clientIp
| sort hits desc
| limit 25
```

A3) Top requested URIs (what are they trying to reach?)

```
fields @timestamp, httpRequest.uri as uri
| stats count() as hits by uri
| sort hits desc
| limit 25
```

A4) Blocked requests only (who/what is being blocked?)

```
fields @timestamp, action, httpRequest.clientIp as clientIp, httpRequest.uri as uri
| filter action = "BLOCK"
| stats count() as blocks by clientIp, uri
| sort blocks desc
| limit 25
```

A5) Which WAF rule is doing the blocking?

```
fields @timestamp, action, terminatingRuleId, terminatingRuleType
| filter action = "BLOCK"
| stats count() as blocks by terminatingRuleId, terminatingRuleType
| sort blocks desc
| limit 25
```

A6) Rate of blocks over time (did it spike?)

```
fields @timestamp, httpRequest.clientIp as clientIp, httpRequest.uri as uri
| filter uri like /wp-login|xmlrpc|.env|admin|phpmyadmin|.git|/login/i
| stats count() as hits by clientIp, uri
```

```
| sort hits desc
| limit 50
```

A7) Suspicious scanners (common patterns: admin paths, wp-login, etc.)

```
fields @timestamp, httpRequest.clientIp as clientIp, httpRequest.uri as uri
| filter uri like /wp-login|xmlrpc|.env|admin|phpmyadmin|.git|/login/i
| stats count() as hits by clientIp, uri
| sort hits desc
| limit 50
```

A8) Country/geo (if present in your WAF logs)

Some WAF log formats include httpRequest.country. If yours does:

```
fields @timestamp, httpRequest.country as country
| stats count() as hits by country
| sort hits desc
| limit 25
```

B) App Queries (EC2 app log group)

These assume your app logs include meaningful strings like ERROR, DBConnectionErrors, timeout, etc (You should enforce this.)

B1) Count errors over time (this should line up with the alarm window)

```
fields @timestamp, @message
| filter @message like /ERROR|Exception|Traceback|DB|timeout|refused/i
| stats count() as errors by bin(1m)
| sort bin(1m) asc
```

B2) Show the most recent DB failures (trriage view)

```
fields @timestamp, @message
```

```
| filter @message like /DB|mysql|timeout|refused|Access denied|could not connect/i
| sort @timestamp desc
| limit 50
```

B3) “Is it creds or network?” classifier hints

Credentials drift often shows: Access denied, authentication failures

Network/SecurityGroup often shows: timeout, refused, “no route”, hang

```
fields @timestamp, @message
| filter @message like /Access denied|authentication failed|timeout|refused|no route|could not connect/i
| stats count() as hits by
  case(
    @message like /Access denied|authentication failed/i, "Creds/Auth",
    @message like /timeout|no route/i, "Network/Route",
    @message like /refused/i, "Port/SG/ServiceRefused",
    "Other"
  )
| sort hits desc
```

B4) Extract structured fields (Requires log JSON)

If you log JSON like: {"level":"ERROR","event":"db_connect_fail","reason":"timeout"}:

```
fields @timestamp, level, event, reason
| filter level="ERROR"
| stats count() as n by event, reason
| sort n desc
```

(Thou Shalt need to emit JSON logs for this one.)

C) Correlation “Enterprise-style” mini-workflow (Runbook Section)

Add this to the incident runbook:

Step 1 – Confirm signal timing

CloudWatch alarm time window: last 5–15 minutes

Run App B1 to see error spike time bins

Step 2 – Decide: Attack vs Backend Failure

Run WAF A1 + A6:

If BLOCK spikes align with incident time → likely external pressure/scanning

If WAF is quiet but app errors spike → likely backend (RDS/SG/creds)

Step 3 – If backend failure suspected

Run App B2 and classify:

Access denied → secrets drift / wrong password

timeout → SG/routing/RDS down

Then retrieve known-good values:

Parameter Store /lab/db/

Secrets Manager /<prefix>/rds/mysql

Step 4 – Verify recovery

App errors return to baseline (B1)

WAF blocks stabilize (A6)

Alarm returns to OK

curl https://app.chewbacca-growl.com/list works

Good news — **Bonus F requires NO new Terraform resources, variables, or outputs.** It's a pure documentation/runbook bonus. The queries are run manually in CloudWatch Logs Insights during incident response.

Here's the breakdown:

Lab 1C Bonus F: CloudWatch Logs Insights Query Pack

What This Bonus Is

This is a **runbook addition**, not infrastructure. You'll document and practice queries for incident response.

Prerequisites Already Complete

- WAF logging to CloudWatch (`aws-waf-logs-helga-webacl01`) — done in Bonus E
- App log group (`helga_log_group01`) — exists from base lab
- ALB logs to S3 — done in Bonus D (queried via S3/Athena, not Logs Insights)

Step 1: Identify Your Log Groups

Run this to confirm your log group names:

```
aws logs describe-log-groups --query "logGroups[*].logGroupName" --output table
```

Expected:

- `aws-waf-logs-helga-webacl01` (WAF logs)
- Something like `/aws/ec2/helga-rds-app` or `helga-log-group01` (App logs)

```
brigh@DESKTOP-62DEPP9 MINGW64 ~/Documents/TheoWAF/class7/AWS/Classes/armageddon_3 (master)
$ aws logs describe-log-groups --query "logGroups[*].logGroupName" --output table
-----
| DescribeLogGroups |
+-----+
| /aws/ec2/helga-rds-app |
| /aws/ec2/lab-rds-app   |
| /helga/ec2/user-data   |
| /helga/notes-app       |
| aws-waf-logs-helga-webacl01 |
+-----+
```

Step 2: Test WAF Queries (AWS Console or CLI)

Option A: AWS Console

1. Go to CloudWatch → Logs Insights
2. Select log group: `aws-waf-logs-helga-webacl01`

3. Set time range: Last 15 minutes
4. Paste and run queries from section A of the instructions

Option B: CLI

```
aws logs start-query \  
  --log-group-name aws-waf-logs-helga-webacl01 \  
  --start-time $(date -d '15 minutes ago' +%s) \  
  --end-time $(date +%s) \  
  --query-string 'fields @timestamp, action | stats count() a  
s hits by action | sort hits desc'
```

Then get results:

```
aws logs get-query-results --query-id <QUERY_ID>
```

```
aws logs get-query-results --query-id 3e4f0fe5-1da1-49ef-9b91  
-68a9353765cf
```

```
brigh@DESKTOP-62DEPP9 MINGW64 ~/Documents/TheoWAF/class7/AWS/Classes/armageddon_3 (master)
$ aws logs get-query-results --query-id 3e4f0fe5-1da1-49ef-9b91-68a9353765cf
{
  "queryLanguage": "CWLI",
  "results": [
    [
      {
        "field": "action",
        "value": "ALLOW"
      },
      {
        "field": "hits",
        "value": "40"
      }
    ],
    [
      {
        "field": "action",
        "value": "BLOCK"
      },
      {
        "field": "hits",
        "value": "1"
      }
    ]
  ],
  "statistics": {
    "recordsMatched": 41.0,
    "recordsScanned": 41.0,
    "estimatedRecordsSkipped": 0.0,
  }
}
```

```
brigh@DESKTOP-62DEPP9 MINGW64 ~/Documents/TheoWAF/class7/AWS/Classes/armageddon_3 (master)
$ aws logs get-query-results --query-id 3e4f0fe5-1da1-49ef-9b91-68a9353765cf
{
  "field": "action",
  "value": "ALLOW"
},
{
  "field": "hits",
  "value": "40"
}
],
[
  {
    "field": "action",
    "value": "BLOCK"
  },
  {
    "field": "hits",
    "value": "1"
  }
]
],
"statistics": {
  "recordsMatched": 41.0,
  "recordsScanned": 41.0,
  "estimatedRecordsSkipped": 0.0,
  "bytesScanned": 67317.0,
  "estimatedBytesSkipped": 0.0,
  "logGroupsScanned": 1.0
},
"status": "Complete"
}
```

Step 3: Test App Log Queries

Same process, but target your app log group:

```
brigh@DESKTOP-62DEPP9 MINGW64 ~/Documents/TheoWAF/class7/AWS/Classes/armageddon_3 (master)
$ aws logs describe-log-groups --query "logGroups[?contains(logGroupName, 'helga')].logGroupName" --output table
-----
| DescribeLogGroups |
+-----+
| /aws/ec2/helga-rds-app |
| /helga/ec2/user-data |
| /helga/notes-app |
| aws-waf-logs-helga-webacl01 |
+-----+
```

```
aws logs describe-log-groups --query "logGroups[?contains(log
```

```
GroupName, 'helga')].logGroupName" --output table
```

```
aws logs start-query \  
  --log-group-name <YOUR_APP_LOG_GROUP> \  
  --start-time $(date -d '15 minutes ago' +%s) \  
  --end-time $(date +%s) \  
  --query-string 'fields @timestamp, @message | filter @message like /ERROR|Exception|timeout/i | sort @timestamp desc | limit 50'
```

```
aws logs start-query \  
  --log-group-name /aws/ec2/helga-rds-app \  
  --start-time $(date -d '15 minutes ago' +%s) \  
  --end-time $(date +%s) \  
  --query-string 'fields @timestamp, @message | filter @message like /ERROR|Exception|timeout/i | sort @timestamp desc | limit 50'
```

```
aws logs start-query \  
  --log-group-name /helga/ec2/user-data \  
  --start-time $(date -d '15 minutes ago' +%s) \  
  --end-time $(date +%s) \  
  --query-string 'fields @timestamp, @message | filter @message like /ERROR|Exception|timeout/i | sort @timestamp desc | limit 50'
```

```
aws logs start-query \  
  --log-group-name /helga/notes-app \  
  --start-time $(date -d '15 minutes ago' +%s) \  
  --end-time $(date +%s) \  
  --query-string 'fields @timestamp, @message | filter @message
```

```
ge like /ERROR|Exception|timeout/i | sort @timestamp desc | l
imit 50'
```

```
aws logs start-query \  
  --log-group-name aws-waf-logs-helga-webacl01 \  
  --start-time $(date -d '15 minutes ago' +%s) \  
  --end-time $(date +%s) \  
  --query-string 'fields @timestamp, @message | filter @messa  
ge like /ERROR|Exception|timeout/i | sort @timestamp desc | l  
imit 50'
```

Step 4: Document in Your Runbook

Create a runbook page (or add to existing) with:

1. Your specific log group names
2. The query pack from the instructions (adapted with `helga` naming)
3. The correlation workflow (Steps 1-4 from section C)

Verification

Checkpoint	How to Verify
WAF queries return data	Run query A1, see ALLOW/BLOCK counts
App queries work	Run query B1 or B2 (may be empty if no errors)
Know your log groups	<code>aws logs describe-log-groups</code> lists them

What You DON'T Need to Do

-  No new `.tf` files
-  No new variables
-  No new outputs
-  No `terraform apply`

Using the AWS Console for Logs Insights (Recommended)

The console is faster for incident response and avoids CLI escaping issues.

Step-by-Step: Console Queries

1. Open CloudWatch Logs Insights

- Go to AWS Console → CloudWatch → Logs → Logs Insights
- Or direct link: `https://us-east-2.console.aws.amazon.com/cloudwatch/home?region=us-east-2#logsV2:logs-insights`

2. Select Your Log Group

- For WAF queries: Select `aws-waf-logs-helga-webacl01`
- For App queries: Select `/helga/notes-app`

3. Set Time Range

- Click the time dropdown (top right)
- Select "Last 15 minutes" (or match your incident window)

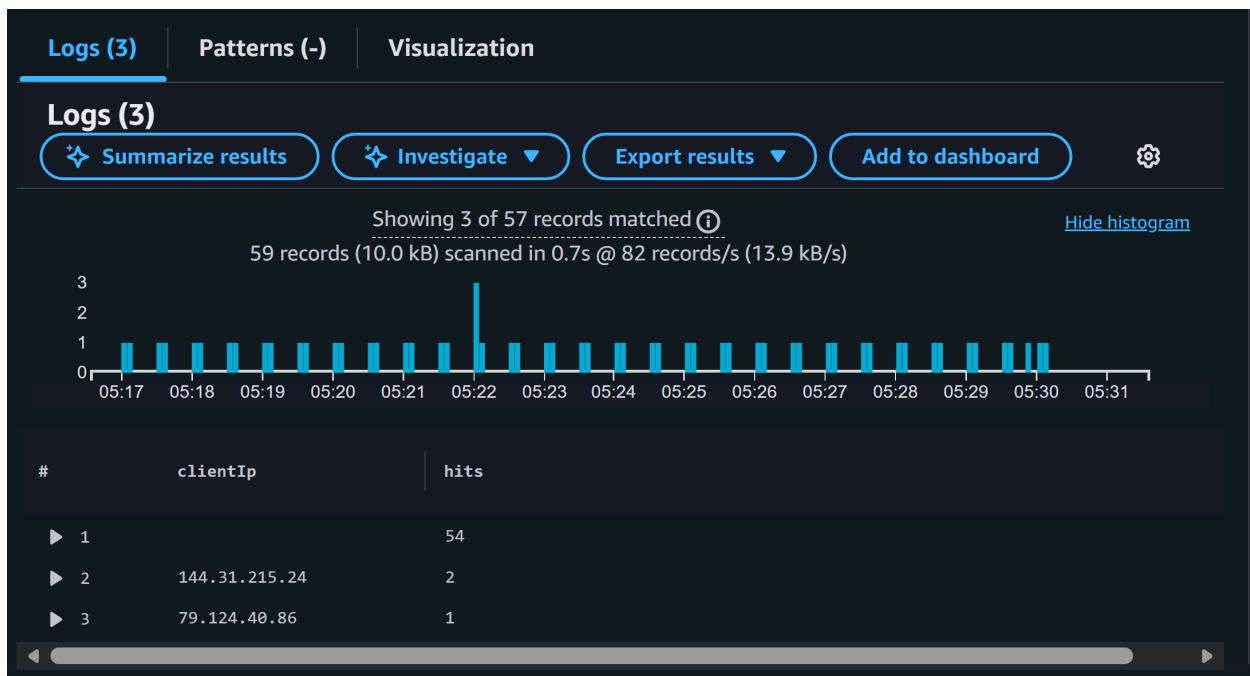
4. Paste and Run Queries

WAF Query A1 — What's happening right now?

```
fields @timestamp, action
| stats count() as hits by action
| sort hits desc
```

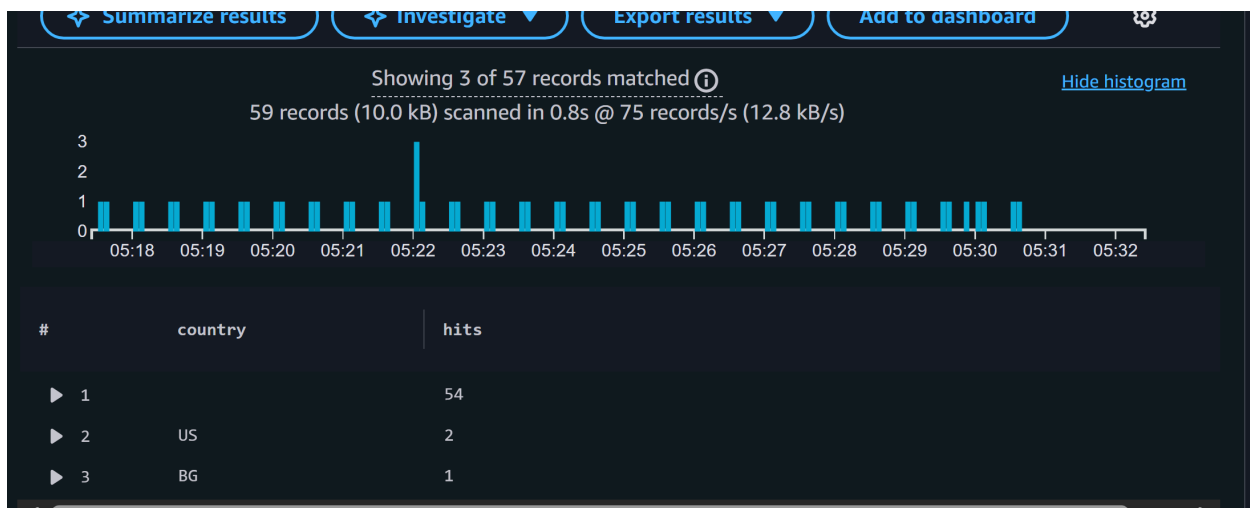
WAF Query A2 — Top client IPs

```
fields @timestamp, httpRequest.clientIp as clientIp
| stats count() as hits by clientIp
| sort hits desc
| limit 25
```



WAF Query A8 — Country breakdown

```
fields @timestamp,
```



App Query B1 — Errors over time

```
fields @timestamp, @message
| filter @message like /ERROR|Exception|Traceback|DB|timeout|
```

```

refused/i
| stats count() as errors by bin(1m)
| sort bin(1m) asc

```

App Query B2 — Recent DB failures

```

fields @timestamp, @message
| filter @message like /DB|mysql|timeout|refused|Access denied|could not connect/i
| sort @timestamp desc
| limit 50

```



5. Click "Run query"

Git Bash CLI Workaround (MSYS_NO_PATHCONV)

When using Git Bash on Windows with log groups that start with `/`, you must disable path conversion:

```

MSYS_NO_PATHCONV=1 aws logs start-query --log-group-name "/he
lga/notes-app" --start-time $(date -d '15 minutes ago' +%s) -
-end-time $(date +%s) --query-string "fields @timestamp, @mes
sage | sort @timestamp desc | limit 50"

```

Then get results:

```

aws logs get-query-results --query-id <QUERY_ID>

```

Why this is needed: Git Bash automatically converts paths starting with `/` to Windows paths (e.g., `/helga/notes-app` becomes `C:/Program Files/Git/helga/notes-app`). The `MSYS_NO_PATHCONV=1` prefix disables this behavior for that command.

✅ Bonus F Complete — Summary

Completed: January 17, 2026

Objectives Achieved

Objective	Status
Identify all log groups	✅ Complete
Run WAF queries (A1, A2, A8)	✅ Verified
Run App queries (B1, B2)	✅ Verified
Document Git Bash workaround	✅ Complete

Log Groups Confirmed

- **WAF logs:** `aws-waf-logs-helga-webacl01`
- **App logs:** `/helga/notes-app`
- **User data logs:** `/helga/ec2/user-data`

Key Learnings

1. **Git Bash path conversion** — Use `MSYS_NO_PATHCONV=1` prefix for log groups starting with `/`
2. **Regex syntax** — Use `(?i)` at the start for case-insensitive matching, not `/i` at the end
3. **Console vs CLI** — AWS Console is faster for incident response queries with complex regex

Result: CloudWatch Logs Insights query pack documented and tested. Ready for incident response.

📘 Lab 1C Incident Response Runbook

Project: helga

Region: us-east-2

Account: 919113286081

Last Updated: January 17, 2026

Environment Reference

Resource	Value
WAF Log Group	aws-waf-logs-helga-webacl01
App Log Group	/helga/notes-app
User Data Log Group	/helga/ec2/user-data
ALB Access Logs	S3: helga-alb-logs-bucket
Application URL	https://app.helga-growl.com
CloudWatch Console	Logs Insights

Step 1: Confirm Signal Timing

When an alarm fires:

1. Note the CloudWatch alarm time window (last 5–15 minutes)
2. Open CloudWatch Logs Insights
3. Select log group: /helga/notes-app
4. Run **App Query B1** to see error spike time bins:

```
fields @timestamp, @message
| filter @message like /ERROR|Exception|Traceback|DB|timeout|
refused/
| stats count() as errors by bin(1m)
| sort bin(1m) asc
```

Step 2: Decide — Attack vs Backend Failure

Run WAF Query A1 (select aws-waf-logs-helga-webacl01):

```
fields @timestamp, action
| stats count() as hits by action
| sort hits desc
```

Run WAF Query A6 (rate of blocks over time):

```
fields @timestamp, action
| filter action = "BLOCK"
| stats count() as blocks by bin(1m)
| sort bin(1m) asc
```

Decision Matrix:

WAF Blocks	App Errors	Likely Cause
⬆ Spiking	Normal	External attack/scanning
Normal	⬆ Spiking	Backend failure (RDS/SG/creds)
⬆ Spiking	⬆ Spiking	Attack causing backend stress

Step 3: If Backend Failure Suspected

Run App Query B2 (select `/helga/notes-app`):

```
fields @timestamp, @message
| filter @message like /DB|mysql|timeout|refused|Access denied|could not connect/
| sort @timestamp desc
| limit 50
```

Classify the error:

Error Pattern	Likely Cause	Action
<code>Access denied</code> / <code>authentication failed</code>	Secrets drift / wrong password	Check Secrets Manager
<code>timeout</code> / <code>no route</code>	Network/routing/SG issue	Check Security Groups, RDS status

Error Pattern	Likely Cause	Action
<code>refused</code> / <code>could not connect</code>	Port blocked or service down	Check RDS status, SG rules

Retrieve known-good values:

```
# Parameter Store
aws ssm get-parameters-by-path --path "/helga/db" --with-decryption

# Secrets Manager
aws secretsmanager get-secret-value --secret-id helga/rds/mysql
```

Step 4: Verify Recovery

Check	Command / Action	Expected Result
App errors baseline	Re-run App Query B1	Error count returns to 0 or near-0
WAF blocks stable	Re-run WAF Query A6	No unusual spikes
Alarm status	CloudWatch → Alarms	State = OK
App functional	<code>curl https://app.helga-growl.com/list</code>	Returns valid response

Quick Reference: All Queries

▼ **WAF Queries** (Log group: `aws-waf-logs-helga-webacl01`)

A1 — Current action breakdown:

```
fields @timestamp, action
| stats count() as hits by action
| sort hits desc
```

A2 — Top client IPs:

```
fields @timestamp, httpRequest.clientIp as clientIp
| stats count() as hits by clientIp
| sort hits desc
| limit 25
```

A4 — Blocked requests detail:

```
fields @timestamp, action, httpRequest.clientIp as clientIp,
httpRequest.uri as uri
| filter action = "BLOCK"
| stats count() as blocks by clientIp, uri
| sort blocks desc
| limit 25
```

A5 — Which rule is blocking:

```
fields @timestamp, action, terminatingRuleId, terminatingRule
Type
| filter action = "BLOCK"
| stats count() as blocks by terminatingRuleId, terminatingRu
leType
| sort blocks desc
```

A8 — Country breakdown:

```
fields @timestamp, httpRequest.country as country
| stats count() as hits by country
| sort hits desc
| limit 25
```

▼ App Queries (Log group: `/helga/notes-app`)

B1 — Errors over time:

```
fields @timestamp, @message
| filter @message like /ERROR|Exception|Traceback|DB|timeout|
refused/
| stats count() as errors by bin(1m)
| sort bin(1m) asc
```

B2 — Recent DB failures:

```
fields @timestamp, @message
| filter @message like /DB|mysql|timeout|refused|Access denied|could not connect/
| sort @timestamp desc
| limit 50
```

B3 — Error classifier:

```
fields @timestamp, @message
| filter @message like /Access denied|authentication failed|timeout|refused|no route|could not connect/
| stats count() as hits by
  case(
    @message like /Access denied|authentication failed/, "Creds/Auth",
    @message like /timeout|no route/, "Network/Route",
    @message like /refused/, "Port/SG/ServiceRefused",
    "Other"
  )
| sort hits desc
```

Runbook Metadata

Author: Willie Bright

Created: January 17, 2026

Lab: Armageddon Challenge — Lab 1C Bonus F

Status:  Complete and tested